

# Workshop „Programmieren“

## in Python · Arbeitsauftrag

Fachinformatiker\*in für Daten- und Prozessanalyse

1. Ausbildungsjahr

**5**

Tage Workshop

**3**

Teams

**1**

eigenes Programm pro  
Team

Name, Vorname

Team

Unser Projekt

Mein Feature

### Hinweise

Dieses Heft führt euch durch die Workshop-Woche. Die Teile 1 bis 5 gehen wir am Montag gemeinsam durch. Die Teile 6 bis 10 begleiten euch Phase für Phase. Im Anhang findet ihr Vorlagen, Werkzeuge und Exkurse zum Nachschlagen.

Den Code zum Workshop (Projektvorlage und Beispiele) findet ihr im GitHub-Repository **Workshop**. Wie ihr es holt, steht in Anhang F. Das Heft umfasst **32 Seiten** einschließlich dieses Deckblatts.

### Inhalt

#### WOCHE

|    |                                       |    |
|----|---------------------------------------|----|
| 1  | Worum es geht                         | 2  |
| 2  | Ablauf der Woche                      | 3  |
| 3  | Regeln und Rollen im Team             | 4  |
| 4  | Woran wir ein gutes Ergebnis erkennen | 5  |
| 5  | Checkliste für eure Lösung            | 6  |
| 6  | Phase 1: Ideen finden                 | 7  |
| 7  | Phase 2: Planen                       | 10 |
| 8  | Phase 3: Umsetzen                     | 11 |
| 9  | Zwischenreview                        | 13 |
| 10 | Präsentation und Reflexion            | 14 |

#### ANHANG

|   |                                      |    |
|---|--------------------------------------|----|
| A | Programmieren mit System             | 16 |
| B | Steckbrief und „Mein Feature“        | 18 |
| C | Bogen Zwischenreview                 | 20 |
| D | Bogen Präsentation                   | 21 |
| E | Coding Conventions und Clean Code    | 22 |
| F | Basics: Arbeiten mit GitHub          | 25 |
| G | Bibliotheken: Standardmodule und pip | 27 |
| H | Exkurs: Dateien und Datenströme      | 29 |
| I | Exkurs: Grafische Oberflächen        | 31 |

## TEIL 1 · ZIEL

# Worum es in dieser Woche geht

Ihr entwickelt im Team ein eigenes Python-Programm: ein kleines Werkzeug für die IT-Administration oder ein Spiel. Ihr findet eine Idee, plant, programmiert, holt euch Feedback und stellt euer Programm am Freitag vor.

### AM FREITAG HABT IHR

- ein **lauffähiges Programm**, zu dem jede Person ein eigenes Feature beigetragen hat,
- einen **Steckbrief**, der zeigt, wie ihr geplant habt,
- eine **Präsentation** mit Live-Vorführung, in der jede Person ihren eigenen Code erklärt,
- **Feedback** von den anderen Teams und eine eigene **Reflexion**.

## Das lernst du dabei

### Python-Grundlagen anwenden

Variablen, Datentypen, Verzweigungen, Schleifen, Listen und Funktionen in einem echten Projekt einsetzen.

### Selbstständig kleine Programme erstellen

Dein Feature planst und programmiert du selbst, Schritt für Schritt.

### Vorhandenen Code erweitern

Ihr baut auf einer Projektvorlage auf und verbindet eure Teile zu einem gemeinsamen Programm.

### Sauberen Code schreiben

So, dass andere ihn lesen, verstehen und weiterverwenden können.

### Wichtige Befehle effizient nutzen

Code mit Git holen und aktualisieren, Module einbinden, Pakete installieren.

### Selbstständig und lösungsorientiert handeln

Mit „Programmieren mit System“ (Anhang A) findest du auch dann einen nächsten Schritt, wenn du feststeckst.

### Im Team arbeiten

Aufgaben verteilen, euch gegenseitig helfen, Feedback geben und annehmen.

### UNTERSCHIEDLICHE ERFAHRUNG IST GEWOLLT

Ihr bringt unterschiedlich viel Erfahrung mit. Das ist kein Problem, sondern Absicht. Jede Person wählt ein Feature, das zu ihr passt: **BASIS** **AUFBAU** **PROFI**. Niemand muss alles können. Wichtig ist, dass du dein Feature **verstehst und erklären kannst**.

## Teams

Die Teams stehen fest. Ihr arbeitet in drei Teams mit drei oder vier Personen. Jede Person übernimmt ein eigenes Feature und zusätzlich eine Rolle im Team (Teil 3).

## TEIL 2 · ABLAUF

# Ablauf der Woche

Jede Phase endet mit einem sichtbaren Ergebnis. So wisst ihr jederzeit, wo ihr steht.

| TAG               | ZEIT          | WAS PASSIERT                                                                                                                                           | ERGEBNIS                                                        |
|-------------------|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| <b>Montag</b>     | Vormittag     | <i>Pflicht-Übungsaufgaben zur Klausur</i>                                                                                                              | –                                                               |
|                   | Nachmittag    | <b>Start des Workshops:</b> Arbeitsauftrag gemeinsam durchgehen, Teams und Rollen, woran wir ein gutes Ergebnis erkennen (Teil 1–5)                    | Ihr wisst, was bis Freitag passiert.                            |
|                   | Nachmittag    | <b>Phase 1: Ideen finden</b> (Teil 6)                                                                                                                  | Eure Projektidee steht, Steckbrief Seite 1 ist begonnen.        |
| <b>Dienstag</b>   | 08:45         | <i>Klausur, danach Pause</i>                                                                                                                           | –                                                               |
|                   | 10:05 – 11:45 | <b>Phase 2: Planen</b> (Teil 7): Features festlegen und verteilen, „Mein Feature“ ausfüllen                                                            | Steckbrief fertig und vom Ausbilder angeschaut.                 |
|                   | 11:45 – 12:45 | <i>Mittagspause</i>                                                                                                                                    |                                                                 |
|                   | ab 12:45      | <b>Phase 3: Umsetzen</b> (Teil 8): Projektordner anlegen, erste Teilschritte programmieren                                                             | Jede Person hat einen ersten lauffähigen Teilschritt.           |
| <b>Mittwoch</b>   | Vormittag     | <b>Umsetzen:</b> Muss-Features fertigstellen, Dateien zusammenführen                                                                                   | Das Programm startet und hat die Muss-Features.                 |
|                   | Nachmittag    | <b>Zwischenreview</b> (Teil 9): Je zwei Teams schauen sich gegenseitig Programm und Code an.                                                           | Ihr habt Feedback und wisst, was ihr verbessern wollt.          |
| <b>Donnerstag</b> | ganztags      | <b>Umsetzen:</b> Feedback einarbeiten, Soll-Features, <b>Exkurse für Fortgeschrittene</b> (Anhang G–I), Clean-Code-Durchgang, Präsentation vorbereiten | Das Programm ist fertig, die Präsentation steht.                |
| <b>Freitag</b>    | ganztags      | <b>Präsentation und Vorführung</b> (Teil 10): Jemand aus einem anderen Team bedient euer Programm. Danach Feedback und Reflexion.                      | Ihr habt euer Programm gezeigt und blickt auf die Woche zurück. |

## Jeden Tag

### ZUM START: 5-MINUTEN-STAND-UP

Jede Person beantwortet kurz drei Fragen:

1. Was habe ich seit gestern geschafft?
2. Was nehme ich mir heute vor?
3. Wo hänge ich oder brauche ich Hilfe?

### ZUM SCHLUSS: INTEGRATIONSHECK

1. Alle bringen ihren aktuellen Stand zusammen (Teil 8).
2. `main.py` starten: Läuft alles zusammen?
3. Checkliste (Teil 5) anschauen: Was ist neu erfüllt?
4. Steckbrief aktualisieren: Was ist erledigt, was kommt morgen?

## TEIL 3 · REGELN UND ROLLEN

# Regeln und Rollen im Team

Diese Regeln sorgen dafür, dass jede Person etwas lernt und niemand allein alles macht.

## 1 Jede Person hat ihr eigenes Feature.

In einer eigenen Datei, mit mindestens einer eigenen Funktion. So ist klar, wer wofür zuständig ist.

## 3 Erst planen, dann programmieren.

Bevor du Code schreibst, steht dein Feature als Teilschritte oder Pseudocode auf Papier („Mein Feature“, Anhang B).

## 5 Steckenbleiben ist normal. Aufgeben nicht.

Wenn du nicht weiterkommst, gilt die 10-20-Regel (unten). Es gibt immer einen nächsten Schritt.

## 7 KI ist eine Lernhilfe, kein Lösungsautomat.

Lass dir Dinge erklären, nicht die Lösung schreiben. Was du übernimmst, musst du Zeile für Zeile erklären können. Markiere KI-Hilfe mit einem Kommentar.

## 2 Wer tippt, entscheidet.

An deinem Feature hast du die Tastatur. Die anderen helfen als Navigator: Sie stellen Fragen, statt vorzusagen oder selbst zu tippen.

## 4 Klein anfangen.

Erst Muss, dann Soll, dann Kann. Ein kleines Programm, das läuft, ist mehr wert als ein großes, das nicht läuft.

## 6 Jede Person erklärt ihren eigenen Code.

Am Freitag zeigst du dein Feature selbst. Schreibe deshalb nur Code, den du erklären kannst.

## 8 Feedback ist freundlich und konkret.

Sprecht über Code, nicht über Menschen. „Den Namen `x` verstehe ich nicht“ statt „Das ist schlecht“.

## Die 10-20-Regel: Wenn du feststeckst

### 0 bis 10 Minuten: allein

Geh die Schritte aus „Programmieren mit System“ durch (Anhang A). Mach das Problem kleiner, teste eine Vermutung mit `print()`.



### Nach 10 Minuten: Team fragen

Frag mit der Hilfe-Formel: „Ich möchte X erreichen. Ich habe Y versucht. Ich erwarte A, bekomme aber B.“



### Nach 20 Minuten: Ausbilder fragen

Kommt ihr auch zusammen nicht weiter, holt ihr den Ausbilder dazu. Das ist kein Scheitern, sondern effizient.

## Rollen im Team

Zusätzlich zu ihrem Feature übernimmt jede Person eine Rolle. In Dreierteams übernimmt eine Person zwei Rollen.

### PLANUNG

Hält den Steckbrief aktuell und achtet darauf, dass erst die Muss-Features fertig werden.

### TEST

Probiert die Features der anderen aus, auch mit falschen Eingaben, und meldet Fehler an die zuständige Person.

### INTEGRATION

Sammelt die Dateien, baut die Features in `main.py` ein und startet täglich das ganze Programm.

### MODERATION

Leitet Stand-up und Integrationscheck, achtet auf die Zeit und organisiert die Präsentation.

## TEIL 4 · ERGEBNIS

# Woran wir ein gutes Ergebnis erkennen

Diese Merkmale beschreiben, wohin ihr arbeitet. Sie sind die Grundlage für das Feedback im Zwischenreview und nach der Präsentation.

### KEINE PUNKTE, KEINE NOTEN

Im Workshop wird nichts benotet. Alle Rückmeldungen sind **qualitativ**: Was ist gelungen, was könnt ihr noch verbessern? Größer ist dabei nicht besser. Ein kleines Programm, das sauber läuft und das alle erklären können, ist ein sehr gutes Ergebnis.

1

#### Es läuft.

Das Programm startet, die Muss-Features funktionieren, und es stürzt bei einer falschen Eingabe nicht ab.

Leitfrage: Was passiert, wenn ich im Menü „x“ eingebe?

2

#### Man kann es bedienen.

Texte und Menü sind verständlich. Eine Person aus einem anderen Team kommt ohne eure Hilfe zurecht.

Leitfrage: Weiß ich als fremde Person jederzeit, was ich eingeben soll?

3

#### Die Seminarinhalte sind sichtbar.

Ihr setzt ein, was ihr im Seminar gelernt habt. Die Checkliste in Teil 5 zeigt, was dazugehört.

Leitfrage: Wo im Code steht die for-Schleife, wo die eigene Funktion?

4

#### Der Code ist sauber.

Namen sind verständlich, Funktionen erledigen jeweils eine Aufgabe, nichts ist unnötig doppelt (Anhang E).

Leitfrage: Könnte jemand aus einem anderen Team ein Feature erweitern?

5

#### Jede Person hat ihren Teil.

Jede Person hat ein erkennbares Feature beigetragen und kann es zeigen und erklären.

Leitfrage: Was macht diese Funktion, und warum steht diese Zeile hier?

6

#### Das Vorgehen ist nachvollziehbar.

Steckbrief, Teilschritte und Tests zeigen, wie ihr vorgegangen seid. Feedback aus dem Zwischenreview habt ihr aufgegriffen.

Leitfrage: Was habt ihr nach dem Zwischenreview geändert?

7

#### Ihr habt als Team gearbeitet.

Absprachen eingehalten, Hilfe gegeben und angenommen, Rollen ausgefüllt.

Leitfrage: Wie habt ihr entschieden, wer was macht?

## TEIL 5 · CHECKLISTE

# Checkliste für eure Lösung

Das gehört in euer Team-Programm. Nicht jede Person muss alles einbauen. Tragt ein, wo es steht, das hilft im Zwischenreview und bei der Präsentation.

| MERKMAL                                                                                                                                                                                                          | WO STEHT ES? (DATEI / FUNKTION) |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|
| <b>PYTHON-GRUNDLAGEN AUS DEM SEMINAR</b>                                                                                                                                                                         |                                 |
| <input type="checkbox"/> <b>Variablen</b> mit verständlichen Namen speichern die Werte, die das Programm braucht.                                                                                                |                                 |
| <input type="checkbox"/> Verschiedene <b>Datentypen</b> sind sinnvoll eingesetzt: <code>int</code> , <code>float</code> , <code>str</code> , <code>bool</code> .                                                 |                                 |
| <input type="checkbox"/> <b>Eingaben</b> mit <code>input()</code> werden bei Bedarf umgewandelt, z. B. mit <code>int()</code> oder <code>float()</code> .                                                        |                                 |
| <input type="checkbox"/> <b>Ausgaben</b> mit <code>print()</code> sind verständlich formuliert.                                                                                                                  |                                 |
| <input type="checkbox"/> <b>Rechnen oder Texte verarbeiten</b> , z. B. mit <code>+</code> , <code>*</code> , <code>//</code> , <code>%</code> oder <code>str()</code> .                                          |                                 |
| <input type="checkbox"/> <b>Vergleiche</b> ( <code>==</code> , <code>!=</code> , <code>&lt;</code> , <code>&gt;=</code> ...) und <b>Verknüpfungen</b> ( <code>and</code> , <code>or</code> , <code>not</code> ). |                                 |
| <input type="checkbox"/> <b>Verzweigung</b> mit <code>if</code> , <code>elif</code> und <code>else</code> .                                                                                                      |                                 |
| <input type="checkbox"/> <b>while-Schleife</b> , z. B. für das Menü oder um eine falsche Eingabe zu wiederholen.                                                                                                 |                                 |
| <input type="checkbox"/> <b>for-Schleife</b> mit <code>range()</code> oder über eine Liste.                                                                                                                      |                                 |
| <input type="checkbox"/> <b>Liste</b> : anlegen, mit <code>append()</code> erweitern, mit <code>len()</code> zählen, durchlaufen.                                                                                |                                 |
| <input type="checkbox"/> <b>Eigene Funktionen</b> mit Parametern und <code>return</code> . Jede Person schreibt mindestens eine.                                                                                 |                                 |
| <input type="checkbox"/> Mindestens ein <b>Modul</b> ist eingebunden und genutzt, z. B. <code>random</code> , <code>math</code> oder <code>time</code> (Anhang G).                                               |                                 |
| <b>AUFBAU UND QUALITÄT</b>                                                                                                                                                                                       |                                 |
| <input type="checkbox"/> <code>main.py</code> zeigt ein Menü. Jedes Feature liegt in einer <b>eigenen Datei</b> .                                                                                                |                                 |
| <input type="checkbox"/> Jede Feature-Datei lässt sich auch <b>einzeln starten und testen</b> .                                                                                                                  |                                 |
| <input type="checkbox"/> Bei einer <b>falschen Eingabe</b> stürzt das Programm nicht ab, sondern fragt erneut.                                                                                                   |                                 |
| <input type="checkbox"/> Mindestens eine Funktion ist mit <b>Schreibtischtest oder Debugger</b> überprüft.                                                                                                       |                                 |
| <input type="checkbox"/> Der <b>Clean-Code-Check</b> aus Anhang E ist für jede Datei gemacht.                                                                                                                    |                                 |
| <b>VORGEHEN</b>                                                                                                                                                                                                  |                                 |
| <input type="checkbox"/> Der <b>Steckbrief</b> ist ausgefüllt und vom Ausbilder angeschaut.                                                                                                                      |                                 |
| <input type="checkbox"/> Jede Person hat ihr Feature mit <b>Teilschritten oder Pseudocode</b> geplant.                                                                                                           |                                 |
| <input type="checkbox"/> Mindestens ein Punkt aus dem <b>Zwischenreview</b> ist umgesetzt.                                                                                                                       |                                 |
| <input type="checkbox"/> Die <b>Präsentation</b> ist vorbereitet. Jede Person kann ihren Code erklären.                                                                                                          |                                 |
| <b>ZUSATZ FÜR FORTGESCHRITTENE (FREIWILLIG)</b>                                                                                                                                                                  |                                 |
| <input type="checkbox"/> Daten werden in einer <b>Datei</b> gespeichert und wieder geladen (Anhang H).                                                                                                           |                                 |
| <input type="checkbox"/> Ein Teil hat eine <b>grafische Oberfläche</b> mit <code>tkinter</code> (Anhang I).                                                                                                      |                                 |
| <input type="checkbox"/> Ein <b>externes Paket</b> ist per <code>pip</code> installiert und genutzt (Anhang G).                                                                                                  |                                 |

## TEIL 6 · PHASE 1 MONTAG, NACHMITTAG

# Phase 1: Ideen finden

Ihr entscheidet euch für ein Admin-Werkzeug oder ein Spiel. Auf den nächsten beiden Seiten findet ihr konkrete Vorschläge. Eine eigene Idee ist ausdrücklich willkommen.

## 1 Allein sammeln 5 min

Jede Person notiert für sich drei Ideen. Schaut dafür in den Ideenpool oder überlegt, was euch im Alltag nützen oder Spaß machen würde.

## 2 Vorstellen 15 min

Reihum stellt jede Person ihre Ideen in einem Satz vor. Noch keine Kritik, nur Rückfragen zum Verständnis.

## 3 Prüfen 15 min

Geht mit den Prüffragen (unten) durch die Ideen. Streicht, was nicht passt. Ideen lassen sich auch kombinieren.

## 4 Entscheiden 10 min

Jede Person hat drei Striche und verteilt sie auf die übrigen Ideen. Bei Gleichstand: kurz diskutieren und noch einmal abstimmen.

## 5 Festhalten 10 min

Tragt Projektname und Idee in einem Satz in den Steckbrief ein (Anhang B). Sammelt erste Features für das Menü. Verteilt werden sie erst am Dienstag.

### PRÜFFRAGEN FÜR EINE GUTE IDEE

- Läuft eine einfache Version schon **bis Mittwochmittag**?
- Gibt es für **jede Person ein eigenes Feature**, auf ihrer Stufe?
- Lässt es sich als **Menü mit mehreren Punkten** bauen?
- Kommen **Seminarinhalte** darin vor (Checkliste, Teil 5)?
- Habt ihr **Lust** darauf?

### EIN MENÜ VERBINDET ALLES

Fast jede Idee lässt sich so aufbauen: `main.py` zeigt ein Menü, und jeder Menüpunkt ist ein Feature einer Person.

Beispiel Passwort-Werkstatt:

- 1 – Passwort erzeugen **BASIS**
- 2 – Passwort prüfen **AUFBAU**
- 3 – Merksatz-Passwort **AUFBAU**
- 4 – Passwörter speichern **PROFI**

## TEIL 6 · IDEENPOOL

# Ideenpool: Admin-Werkzeuge

Kleine Helfer, wie man sie in der IT wirklich braucht. Die Features sind Beispiele, ihr dürft sie ändern und ergänzen.

## ADMIN-WERKZEUG

## Passwort-Werkstatt

Sichere Passwörter erzeugen und prüfen.

## BASIS

Passwort mit fester Länge aus zufälligen Zeichen erzeugen

## AUFBAU

Passwort prüfen: lang genug, Ziffer, Großbuchstabe? Mit Begründung

## PROFI

Erzeugte Passwörter mit Zweck in einer Datei speichern

## ADMIN-WERKZEUG

## Geräteverleih

Wer hat welchen Laptop, welches Headset?

## BASIS

Alle Geräte einer Liste nummeriert anzeigen

## AUFBAU

Gerät ausleihen und zurückgeben, Status „frei“ oder „verliehen“

## PROFI

Ausleihe speichern, Rückgabedatum mit `datetime`

## ADMIN-WERKZEUG

## Benutzerkonten-Generator

Für neue Kolleginnen und Kollegen Konten vorbereiten.

## BASIS

Aus Vor- und Nachname einen Benutzernamen bauen, z. B. `max.muster`

## AUFBAU

Doppelte Namen erkennen und durchnummerieren, Startpasswort erzeugen

## PROFI

Namensliste aus einer Datei lesen, Ergebnis in eine Datei schreiben

## ADMIN-WERKZEUG

## Helpdesk-Tickets

Störungsmeldungen erfassen und abarbeiten.

## BASIS

Ticket mit Titel anlegen, alle Tickets anzeigen

## AUFBAU

Priorität und Status („offen“, „erledigt“), nur offene Tickets anzeigen

## PROFI

Tickets speichern, Statistik: wie viele offen, wie viele dringend

## ADMIN-WERKZEUG

## IT-Rechner

Rechnen mit Speichergrößen und Datenraten.

## BASIS

Speichergrößen umrechnen: Byte, KB, MB, GB

## AUFBAU

Downloadzeit aus Dateigröße und Bandbreite berechnen

## PROFI

Zahlen zwischen Dezimal und Binär umrechnen, ohne `bin()`

## ADMIN-WERKZEUG

## Inventarliste

Verbrauchsmaterial im Blick behalten.

## BASIS

Artikel mit Menge anzeigen und eine Menge ändern

## AUFBAU

Warnung, wenn ein Bestand unter einen Mindestwert fällt

## PROFI

Bestand speichern und laden, Einkaufsliste erzeugen

## TEIL 6 · IDEENPOOL

# Ideenpool: Spiele

Spiele für die Konsole. Auch hier gilt: Erst eine einfache Version, die läuft, dann ausbauen.

## SPIEL

## Galgenmännchen

Ein Wort Buchstabe für Buchstabe erraten.

## BASIS

Festes Wort, Buchstaben raten, erratene Buchstaben anzeigen

## AUFBAU

Zufallswort aus einer Liste, begrenzte Fehlversuche, Galgen als Text

## PROFI

Wörter aus einer Datei laden, Schwierigkeitsstufen

## SPIEL

## IT-Quiz

Fragen rund um Hardware, Netzwerk und Python.

## BASIS

Fragen nacheinander stellen, Antwort prüfen, richtig oder falsch melden

## AUFBAU

Zufällige Reihenfolge, Auswertung am Ende in Prozent

## PROFI

Fragen aus einer Datei, Kategorien wählbar, Highscore

## SPIEL

## Würfelduell

Zwei Personen würfeln gegeneinander.

## BASIS

Beide würfeln einmal, die höhere Zahl gewinnt

## AUFBAU

Mehrere Runden, Punktestand, Sonderregel bei einem Pasch

## PROFI

Mehrere Würfel mit Kniffel-Kategorien, Highscore-Datei

## SPIEL

## Schere, Stein, Papier

Gegen den Computer, mit Statistik.

## BASIS

Eine Runde gegen den Computer mit `random.choice()`

## AUFBAU

„Best of 5“ mit Spielstand und Statistik am Ende

## PROFI

Erweiterung um Echse und Spock, Oberfläche mit Knöpfen

## SPIEL

## Textadventure

Durch Räume laufen, Gegenstände finden, ein Ziel erreichen.

## BASIS

Ein Raum mit Beschreibung und zwei Entscheidungen

## AUFBAU

Mehrere Räume, Inventar als Liste, Schleife bis zum Ziel

## PROFI

Spielstand speichern und später weiterspielen

## SPIEL

## Mastermind

Einen geheimen Zahlencode knacken.

## BASIS

Geheimen dreistelligen Code erzeugen, Tipp einlesen

## AUFBAU

Rückmeldung: wie viele Ziffern an der richtigen Stelle

## PROFI

Auch „richtige Ziffer, falsche Stelle“ zählen, Versuche begrenzen

### EIGENE IDEE?

Gern. Prüft sie mit den Prüffragen aus Phase 1. Wenn ihr für jede Person ein Feature findet und eine einfache Version bis Mittwochmittag laufen kann, legt los.

## TEIL 7 · PHASE 2 DIENSTAG, 10:05 – 11:45

# Phase 2: Planen

Heute wird noch nicht programmiert. Ihr legt fest, was euer Programm können soll und wer was baut. Grundlage ist der Steckbrief in Anhang B.

## 1 Features sammeln und einordnen

20 min

**Muss** läuft bis Mittwochmittag, **Soll** bis Donnerstag, **Kann** nur, wenn Zeit bleibt.

## 2 Features und Rollen verteilen

10 min

Jede Person übernimmt ein Muss-Feature. Wählt ehrlich: Ein Basis-Feature, das läuft, bringt mehr als ein Profi-Feature, das nicht fertig wird.

## 3 Das eigene Feature verstehen

15 min

Jede Person füllt „Mein Feature“ aus (Anhang B): Was tut es in einem Satz? Welche Eingabe, welche Ausgabe, welches Beispiel?

## 4 Die Schnittstelle festlegen

10 min

Dateiname, Funktionsname, Parameter, Rückgabewert: So weiß `main.py`, wie es jedes Feature aufruft.

## 5 In Teilschritte zerlegen

30 min

Teilschritte und Pseudocode aufschreiben. Der erste Teilschritt läuft nach 15 bis 30 Minuten.

## 6 Ausbilder schaut drauf

bis 11:45

Zeigt dem Ausbilder euren Steckbrief. Erst danach geht es ans Programmieren.

## Beispiel: „Mein Feature“ ausgefüllt

### PASSWORT ERZEUGEN

BASIS

**In einem Satz:** Erzeugt ein zufälliges Passwort in der gewünschten Länge.

**Eingabe:** Länge als Zahl, z. B. 8

**Ausgabe:** Passwort als Text, z. B. k3T9xQ2a

**Randfall:** Länge 2 ist zu kurz, also erneut fragen.

**Schnittstelle:** Datei `passwort.py`, Funktion `erzeuge_passwort(laenge)`, gibt einen `str` zurück.

### TEILSCHRITTE

- ☐ 1. Ein einzelnes zufälliges Zeichen ausgeben (`random.choice`)
- ☐ 2. Mit einer `for`-Schleife 8 Zeichen aneinanderhängen
- ☐ 3. In eine Funktion mit Parameter `laenge` packen, Ergebnis mit `return`
- ☐ 4. Länge mit `input()` abfragen und umwandeln
- ☐ 5. Zu kurze Länge abfangen und erneut fragen
- ☐ 6. In `main.py` ins Menü einbauen

```
# Pseudocode für Teilschritt 2 und 3
Funktion erzeuge_passwort mit laenge:
    passwort ist zuerst leer
    wiederhole laenge-mal:
        hänge ein zufälliges Zeichen an passwort an
    gib passwort zurück
```

## TEIL 8 · PHASE 3 DIENSTAG 12:45 BIS DONNERSTAG

## Phase 3: Umsetzen

Jetzt wird programmiert, Teilschritt für Teilschritt. Am Mittwochmittag laufen die Muss-Features, am Donnerstag ist das Programm fertig.

### Projektordner anlegen (Dienstag, 12:45)

Holt euch das Repository **Workshop** (Anhang F). Kopiert den Ordner `projektvorlage` an einen eigenen Ort, zum Beispiel nach `Dokumente/workshop-team`. Arbeitet nicht direkt im geklonten Ordner, sonst gibt es Probleme beim Aktualisieren.

```
workshop-team/
├─ main.py           ← Menü, ruft die Features auf
├─ password.py       ← Feature von Person 1
├─ pruefer.py        ← Feature von Person 2
├─ merksatz.py       ← Feature von Person 3
└─ speicher.py       ← Feature von Person 4
```

Jede Person benennt die Datei `feature_vorlage.py` in ihren eigenen Dateinamen um, oder kopiert sie. Die Integrationsrolle baut die Features in `main.py` ein.

Dateinamen: klein, ohne Umlaute und Leerzeichen, z. B. `password.py`. Sie müssen genau zu dem passen, was `main.py` importiert.

### So hängen die Dateien zusammen

`password.py`

```
import random

ZEICHEN = "abcdefghijklmnopqrstuvwxyz23456789"

def erzeuge_passwort(laenge):
    password = ""
    for i in range(laenge):
        password = password + random.choice(ZEICHEN)
    return password

def starte():
    laenge = int(input("Länge: "))
    print("Dein Passwort:", erzeuge_passwort(laenge))

if __name__ == "__main__":
    starte()
```

`main.py` (Ausschnitt)

```
import password

...

if auswahl == "1":
    password.starte()
```

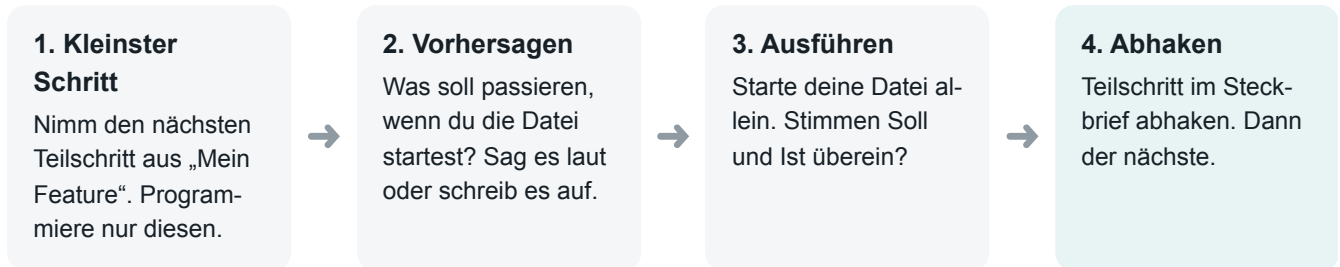
`import password` lädt die Datei `password.py`, die im selben Ordner liegt. Danach ruft `password.starte()` die Funktion aus dieser Datei auf.

`if __name__ == "__main__":` Dieser Block läuft nur, wenn du genau diese Datei startest. So kannst du dein Feature allein testen, ohne dass das Menü der anderen schon fertig sein muss.

## TEIL 8 · PHASE 3

# Arbeiten in kleinen Schritten

Wer in kleinen Schritten programmiert und nach jedem Schritt testet, findet Fehler sofort und kommt schneller ans Ziel.

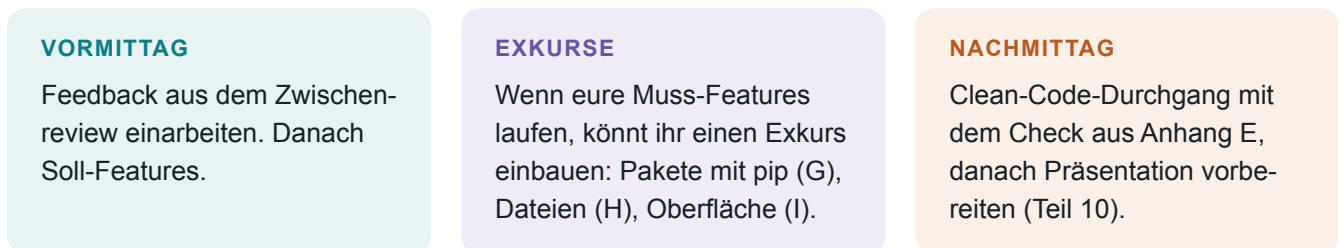


Passt etwas nicht, hilft Schritt 7 „Fehler eingrenzen“ aus Anhang A: Stelle eine Vermutung auf und prüfe sie gezielt mit `print()` oder dem Debugger.

## Mittwochvormittag: Muss-Features fertig

Bis zum Zwischenreview soll euer Programm starten und die Muss-Features zeigen. Was noch nicht fertig ist, nehmt ihr vorübergehend aus dem Menü. Ein Menüpunkt, der abstürzt, hilft niemandem.

## Donnerstag: verbessern, ausbauen, aufräumen



### EXKURSE FÜR FORTGESCHRITTENE

Die Exkurse in Anhang G, H und I gehen über das Seminar hinaus. Sie sind **freiwillig** und nur dann sinnvoll, wenn dein Muss-Feature schon läuft. Jeder Exkurs hat eine Einführung zum Lesen und lauffähige Beispiele im Ordner `beispiele` des Repositorys. Wer einen Exkurs einbaut, erklärt ihn am Freitag kurz den anderen.

## Dateien im Team austauschen

Ihr arbeitet jeweils an eurer eigenen Datei. So entstehen keine Konflikte.

- Legt im Teams-Kanal eures Teams einen Ordner für den Code an. Jede Person lädt **nur ihre eigene Datei** hoch und ersetzt dort die alte Version.
- Die Integrationsrolle lädt beim Integrationscheck alle aktuellen Dateien herunter, legt sie in den gemeinsamen Projektordner und startet `main.py`.
- Ändert nie die Datei einer anderen Person. Gebt stattdessen einen Hinweis.

# Zwischenreview

Je zwei Teams schauen sich gegenseitig Programm und Code an. Ihr bekommt frische Augen auf euer Projekt, und ihr seht, wie andere ein Problem gelöst haben.

## Drei Runden

| RUNDE          | WER REVIEWT WEN?                             | DAS DRITTE TEAM                                      |
|----------------|----------------------------------------------|------------------------------------------------------|
| <b>Runde 1</b> | Team 1 und Team 2 reviewen sich gegenseitig. | Team 3 arbeitet weiter, der Ausbilder schaut vorbei. |
| <b>Runde 2</b> | Team 2 und Team 3 reviewen sich gegenseitig. | Team 1 arbeitet weiter.                              |
| <b>Runde 3</b> | Team 3 und Team 1 reviewen sich gegenseitig. | Team 2 arbeitet weiter.                              |

So reviewt jedes Team zweimal und bekommt von beiden anderen Teams Feedback. Eine Runde dauert etwa 30 Minuten: 15 Minuten in jede Richtung.

## Ablauf in eine Richtung (15 Minuten)

1

### Ausprobieren

5 min

Das Gast-Team bedient das Programm, **ohne Erklärung**. Es denkt dabei laut: „Ich will jetzt ... Ich erwarte ...“. Das Gastgeber-Team schaut zu und schweigt.

2

### Code lesen

5 min

Das Gastgeber-Team zeigt eine Feature-Datei. Das Gast-Team liest sie mit dem Bogen aus Anhang C und stellt Verständnisfragen.

3

### Feedback

5 min

Das Gast-Team gibt Feedback nach dem Muster **zwei Sterne und ein Wunsch**: zwei Dinge, die gut gelungen sind, und ein konkreter Verbesserungsvorschlag.

#### FEEDBACK GEBEN

- Konkret: „Bei Menüpunkt 2 wusste ich nicht, ob ich eine Zahl oder ein Wort eingeben soll.“
- Über die Sache, nicht über die Person.
- Auch Gelungenes benennen, und zwar genau.

#### FEEDBACK ANNEHMEN

- Zuhören und mitschreiben, nicht rechtfertigen.
- Nachfragen, wenn etwas unklar ist.
- Danach im Team entscheiden, was ihr umsetzt.

**Nach dem Review** nehmt ihr euch 10 Minuten im Team: Was nehmen wir uns bis Donnerstag vor? Tragt mindestens einen Punkt unten auf dem Review-Bogen ein und setzt ihn am Donnerstag um.

## TEIL 10 · PRÄSENTATION FREITAG

# Präsentation und Reflexion

Ihr zeigt, was ihr gebaut habt und wie ihr dabei vorgegangen seid. Jedes Team hat etwa 20 Minuten.

## Ablauf pro Team

| TEIL                   | DAUER    | INHALT                                                                                                                                    |
|------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Idee und Plan       | 2 min    | Was macht euer Programm, für wen? Wie habt ihr die Features verteilt?                                                                     |
| 2. Fremde Vorführung   | 5 min    | Eine Person aus einem anderen Team bedient euer Programm. Ihr dürft sagen, was sie ausprobieren soll, aber nicht die Tastatur übernehmen. |
| 3. Code-Einblick       | je 2 min | Jede Person zeigt eine Funktion aus ihrem Feature und erklärt sie Zeile für Zeile.                                                        |
| 4. Rückblick           | 2 min    | Was lief gut? Wo seid ihr steckengeblieben, und wie seid ihr weitergekommen?                                                              |
| 5. Fragen und Feedback | 5 min    | Die Zuhörenden stellen Fragen und geben Feedback aus ihrem Schwerpunkt.                                                                   |

### VORBEREITUNG (DONNERSTAG)

- ☐ Programm läuft auf dem Präsentationsrechner
- ☐ Reihenfolge der Sprechenden steht fest
- ☐ Jede Person hat ihre Funktion ausgewählt
- ☐ Eine Vorführung einmal durchgespielt
- ☐ Plan B, falls etwas abstürzt: Screenshots

### ZUHÖREN MIT SCHWERPUNKT

Jede zuhörende Person bekommt einen Schwerpunkt: **Bedienung**, **Code**, **Erklärung** oder **Zusammenarbeit**. Mit dem Bogen aus Anhang D notiert sie Beobachtungen und stellt danach mindestens eine Frage. Es gibt keine Punkte, nur Beobachtungen und Fragen.

## Reflexion

Zum Abschluss denkt jede Person zuerst allein über die Woche nach (10 Minuten). Danach sagt jede Person in der Abschlussrunde einen Satz: „Das nehme ich mit ...“

WAS HABE ICH DIESE WOCHE GELERNT, DAS ICH VORHER NICHT KONNTE?

---



---

WO BIN ICH STECKENGEBLIEBEN, UND WAS HAT MIR GEHOLFEN WEITERZUKOMMEN?

---



---

WELCHER SCHRITT AUS „PROGRAMMIEREN MIT SYSTEM“ HAT MIR AM MEISTEN GEHOLFEN?

---



---

WAS MACHE ICH BEIM NÄCHSTEN PROJEKT ANDERS?

---



---

## ANHANG

# Anhang: Übersicht

Vorlagen zum Ausfüllen, Werkzeuge und Exkurse zum Nachschlagen.

|          | ANHANG                               | WANN BRAUCHST DU IHN?                                                              | SEITE |
|----------|--------------------------------------|------------------------------------------------------------------------------------|-------|
| <b>A</b> | Programmieren mit System             | Immer, wenn du ein Problem angehst oder feststeckst.                               | 16    |
| <b>B</b> | Steckbrief und „Mein Feature“        | Montag und Dienstag beim Planen, danach als Fortschrittsliste.                     | 18    |
| <b>C</b> | Bogen Zwischenreview                 | Mittwochnachmittag, ein Bogen pro Review.                                          | 20    |
| <b>D</b> | Bogen Präsentation                   | Freitag, ein Bogen pro präsentierendem Team.                                       | 21    |
| <b>E</b> | Coding Conventions und Clean Code    | Beim Programmieren und beim Clean-Code-Durchgang am Donnerstag.                    | 22    |
| <b>F</b> | Basics: Arbeiten mit GitHub          | Dienstag, wenn ihr den Projektordner anlegt.                                       | 25    |
| <b>G</b> | Bibliotheken: Standardmodule und pip | Sobald du ein Modul wie <code>random</code> brauchst. Der pip-Teil ist ein Exkurs. | 27    |
| <b>H</b> | Exkurs: Dateien und Datenströme      | Für Fortgeschrittene, wenn Daten erhalten bleiben sollen.                          | 29    |
| <b>I</b> | Exkurs: Grafische Oberflächen        | Für Fortgeschrittene, wenn ein Feature ein Fenster bekommen soll.                  | 31    |

## VORLAGEN EINZELN DRUCKEN

Steckbrief, „Mein Feature“, die Bögen und die Checkliste liegen im Repository zusätzlich als einzelne PDFs im Ordner `vorlagen`.

# Programmieren mit System

Wenn du feststeckst, hilft ein klarer Ablauf. Jeder Schritt liefert etwas, mit dem du weiterarbeiten kannst.

1

## VERSTEHEN

Was soll passieren?

Formuliere die Aufgabe in eigenen Worten. Kläre Eingabe, gewünschte Ausgabe und offene Fragen.

Notenliste gegeben. Gesucht: der Durchschnitt.

2

## KONKRETISIEREN

Wie sieht ein Beispiel aus?

Lege konkrete Eingaben und erwartete Ergebnisse fest, bevor du Code schreibst.

[2, 3, 1, 4]: Summe 10, Anzahl 4, Ergebnis 2.5.

3

## ZERLEGEN

Welche kleinen Teile gibt es?

Teile das Ziel in überschaubare Teilprobleme. Benenne, welcher Teil noch unklar ist.

Liste prüfen, addieren, zählen, teilen, ausgeben.

4

## PLANEN

Wie löse ich es auf Papier?

Rechne von Hand. Schreibe den Ablauf in Alltagssprache oder Pseudocode auf.

Summe bei 0 starten. Jede Note dazuzählen.

5

## IMPLEMENTIEREN

Was ist der kleinste nächste Schritt?

Programmiere einen Teil, den du bereits erklären kannst. Erweitere ihn anschließend.

Erst eine Note addieren, dann alle in einer Schleife.

6

## TESTEN

Stimmen Soll und Ist überein?

Sage das Ergebnis voraus, führe den Code aus und prüfe auch einen Randfall.

[3] ergibt 3.0. [] soll „Keine Noten“ ergeben.

7

## FEHLER EINGRENZEN

Welche Vermutung kann ich prüfen?

Teste eine Hypothese gezielt, etwa mit print() oder dem Debugger. Ändere eine Sache auf einmal.

Stimmt die Anzahl? Prüfe zuerst len(noten).

8

## REFLEKTIEREN

Welcher Gedanke hat geholfen?

Erkläre deinen Lösungsweg. Halte fest, was du beim nächsten Problem wieder anwenden kannst.

„Die Zwischensummen haben den Fehler gezeigt.“

Du darfst zurückgehen: Ein Test kann zeigen, dass du die Aufgabe neu verstehen oder deinen Plan ändern musst.

## SO BITTEST DU KONKRET UM HILFE

„Ich möchte X erreichen. Ich habe bisher Y versucht. Ich erwarte A, bekomme aber B. Ich vermute, dass es an C liegt. Bei D komme ich nicht weiter.“

# Dein Werkzeugkasten

Wähle die Methode, die zu deiner aktuellen Frage passt. Dein Ziel ist ein kleiner Schritt, der dir neue Klarheit gibt.

## Pseudocode

### Vor dem Programmieren

Beschreibe die Logik in Alltagssprache. So klärst du den Ablauf, bevor du Python-Syntax brauchst.

„Für jede Note: zur bisherigen Summe addieren.“

## Rubber Duck Debugging

### Wenn du den Fehler nicht siehst

Erkläre deinen Code einer Person oder einem Gegenstand laut, Zeile für Zeile. Was erwartest du jeweils?

„Hier setze ich die Summe wieder auf 0. Warum?“

## Minimal Reproducible Example

### Wenn das Problem unübersichtlich ist

Verkleinere Daten und Code, bis nur ein kleines Beispiel übrig bleibt, das denselben Fehler zeigt.

Statt einer ganzen Anwendung: zwei Zahlen und eine Schleife.

## Hypothesengetriebenes Debugging

### Bei unerwartetem Verhalten

Formuliere eine Vermutung. Plane einen gezielten Test und vergleiche die Beobachtung mit deiner Vorhersage.

„Die Anzahl ist falsch.“ Test: `print(len(noten))`.

## Testgetriebenes Denken

### Vor und während der Umsetzung

Lege vorher fest, woran du eine korrekte Lösung erkennst. Prüfe Normalfälle und Grenzen.

[1, 6] soll 3.5 ergeben. Für [] klären wir die Ausgabe.

## DREI REGELN FÜR JEDES PROGRAMMIERPROBLEM

### 01 Mach das Problem kleiner.

Suche einen Teil, den du schon lösen oder gezielt ausprobieren kannst.

### 02 Vermute nicht nur. Experimentiere.

Prüfe deine Hypothese mit möglichst wenig Code und einer klaren Vorhersage.

### 03 Stelle konkrete Fragen.

Beschreibe, was du erreichen willst, was du beobachtest und welche Stelle unklar ist.

Du musst die Lösung noch nicht kennen, um den nächsten Schritt zu finden.

## ANHANG B · VORLAGE

# Team-Steckbrief

Einmal pro Team. Die Planungsrolle hält ihn die ganze Woche aktuell.

TEAM

PROJEKTNAME

UNSERE IDEE IN EINEM SATZ

WER BENUTZT DAS PROGRAMM, UND WOFÜR?

## Mitglieder und Rollen

| NAME | ROLLE (PLANUNG, TEST, INTEGRATION, MODERATION) |
|------|------------------------------------------------|
|      |                                                |
|      |                                                |
|      |                                                |
|      |                                                |

## Features

| NR. | FEATURE | MUSS /<br>SOLL /<br>KANN | STUFE<br>B / A / P | WER | DATEI | FER-<br>TIG              |
|-----|---------|--------------------------|--------------------|-----|-------|--------------------------|
| 1   |         |                          |                    |     |       | <input type="checkbox"/> |
| 2   |         |                          |                    |     |       | <input type="checkbox"/> |
| 3   |         |                          |                    |     |       | <input type="checkbox"/> |
| 4   |         |                          |                    |     |       | <input type="checkbox"/> |
| 5   |         |                          |                    |     |       | <input type="checkbox"/> |
| 6   |         |                          |                    |     |       | <input type="checkbox"/> |
| 7   |         |                          |                    |     |       | <input type="checkbox"/> |

DAS MUSS BIS MITTWOCHMITTAG LAUFEN

☐ Vom Ausbilder angeschaut am \_\_\_\_\_ Kürzel \_\_\_\_\_

## ANHANG B · VORLAGE

# Mein Feature

Einmal pro Person. Erst ausfüllen, dann programmieren.

NAME

FEATURE

STUFE

☐ Basis ☐ Aufbau ☐ Profi

## 1 · Verstehen: Was soll passieren?

MEIN FEATURE IN EINEM SATZ

EINGABE (WAS KOMMT REIN?)

AUSGABE (WAS SOLL RAUSKOMMEN?)

## 2 · Konkretisieren: Beispiele

| EINGABE   | ERWARTETE AUSGABE |
|-----------|-------------------|
|           |                   |
|           |                   |
| Randfall: |                   |

## 3 · Schnittstelle

Datei: \_\_\_\_\_ .py Funktion: def \_\_\_\_\_ ( \_\_\_\_\_ ) :

Gibt zurück: \_\_\_\_\_

## 4 · Zerlegen: Teilschritte

| NR. | TEILSCHRITT (DER ERSTE PASST IN 15 BIS 30 MINUTEN) | FER-TIG                  |
|-----|----------------------------------------------------|--------------------------|
| 1   |                                                    | <input type="checkbox"/> |
| 2   |                                                    | <input type="checkbox"/> |
| 3   |                                                    | <input type="checkbox"/> |
| 4   |                                                    | <input type="checkbox"/> |
| 5   |                                                    | <input type="checkbox"/> |

## 5 · Planen: Pseudocode

## ANHANG C · VORLAGE

# Bogen Zwischenreview

Füllt ihn als Gast-Team aus und gebt ihn danach dem Gastgeber-Team. Es geht um Beobachtungen, nicht um Punkte.

WIR SIND TEAM

WIR REVIEWEN TEAM / PROJEKT

## 1 · Ausprobieren

WAS HAT SOFORT GEKLAPPT?

WO WUSSTEN WIR NICHT WEITER? GAB ES ABSTÜRZE ODER UNERWARTETE AUSGABEN?

## 2 · Code lesen Datei: \_\_\_\_\_

| LEITFRAGE                                                            | UNSERE BEOBACHTUNG |
|----------------------------------------------------------------------|--------------------|
| Verstehen wir die Namen von Variablen und Funktionen ohne Erklärung? |                    |
| Erledigt jede Funktion eine klar erkennbare Aufgabe?                 |                    |
| Gibt es Code, der mehrfach fast gleich vorkommt?                     |                    |
| Welche Inhalte aus der Checkliste (Teil 5) entdecken wir?            |                    |

## 3 · Zwei Sterne und ein Wunsch

★ DAS IST GUT GELUNGEN

★ DAS IST AUCH GUT GELUNGEN

→ UNSER WUNSCH: DAS WÜRDEN WIR VERBESSERN

VOM GASTGEBER-TEAM AUSZUFÜLLEN

DAS SETZEN WIR BIS DONNERSTAG UM

ANHANG D · VORLAGE

# Bogen Präsentation

Ein Bogen pro präsentierendem Team. Achte vor allem auf deinen Schwerpunkt und notiere Beobachtungen. Es gibt keine Punkte.

MEIN NAME

PRÄSENTIERENDES TEAM / PROJEKT

MEIN SCHWERPUNKT

☐ Bedienung ☐ Code ☐ Erklärung ☐ Zusammenarbeit

Leitfragen zu den Schwerpunkten

**BEDIENUNG**

Kam die fremde Person ohne Hilfe zurecht? Wo hat sie gezögert? Was passierte bei falschen Eingaben?

**CODE**

Waren Namen und Funktionen verständlich? Was war besonders geschickt gelöst? Welche Seminarinhalte habe ich entdeckt?

**ERKLÄRUNG**

Konnte jede Person ihre Funktion Zeile für Zeile erklären? Was habe ich dabei selbst gelernt?

**ZUSAMMENARBEIT**

Wurde klar, wer was gebaut hat und wie die Teile zusammenpassen? Wie hat das Team Probleme gelöst?

MEINE BEOBACHTUNGEN

DAS HAT MIR BESONDERS GEFALLEN

MEIN TIPP: DAS WÜRDTE ICH ANDERS MACHEN

MEINE FRAGE AN DAS TEAM

## ANHANG E · WERKZEUG

# Coding Conventions und Clean Code

Code wird viel öfter gelesen als geschrieben, von deinem Team, vom Ausbilder und von dir selbst in zwei Wochen. Einheitliche Regeln machen ihn leichter lesbar.

## Python-Konventionen (PEP 8)

PEP 8 ist der offizielle Stilleitfaden für Python. Die wichtigsten Regeln:

| THEMA                        | REGEL                                                                                           | BEISPIEL                                                        |
|------------------------------|-------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| <b>Einrückung</b>            | Immer 4 Leerzeichen pro Ebene, keine Tabs mischen.                                              | <code>print(x)</code>                                           |
| <b>Variablen, Funktionen</b> | Kleinbuchstaben, Wörter mit Unterstrich verbunden (snake_case).                                 | <code>anzahl_versuche</code><br><code>def zeige_menue():</code> |
| <b>Konstanten</b>            | Werte, die sich nie ändern: Großbuchstaben, oben in der Datei.                                  | <code>MAX_VERSUCHE = 3</code>                                   |
| <b>Umlaute</b>               | In Namen keine Umlaute und kein ß verwenden. In Texten sind sie erlaubt.                        | <code>groesse</code> statt <code>größe</code>                   |
| <b>Leerzeichen</b>           | Um <code>=</code> und Rechenzeichen, nach Kommas. Nicht direkt innerhalb von Klammern.          | <code>summe = a + b</code><br><code>print(a, b)</code>          |
| <b>Leerzeilen</b>            | Zwei Leerzeilen vor jeder Funktion. Innerhalb von Funktionen sparsam, um Abschnitte zu trennen. |                                                                 |
| <b>Imports</b>               | Ganz oben in der Datei, jeder Import in einer eigenen Zeile.                                    | <code>import random</code>                                      |
| <b>Zeilenlänge</b>           | Höchstens etwa 79 Zeichen. Lange Texte lieber aufteilen.                                        |                                                                 |

### PYCHARM HILFT

**Code > Reformat Code** (Windows: Strg + Alt + L, macOS: ⌘ + ⌘ + L) formatiert deine Datei nach PEP 8. Gelbe Wellenlinien zeigen Stilprobleme. Fahr mit der Maus darüber, dann siehst du den Hinweis.

## Docstrings und Kommentare

Ein **Docstring** steht in dreifachen Anführungszeichen direkt unter `def` und sagt in einem Satz, was die Funktion tut. PyCharm zeigt ihn an, wenn jemand die Funktion benutzt. Ein **Kommentar** mit `#` erklärt, warum etwas so gelöst ist.

```
MINDESTLAENGE = 8
```

```
def ist_sicher(password):
    """Gibt True zurück, wenn das Passwort lang genug ist und eine Ziffer enthält."""
    # Länge zuerst prüfen: Das ist die häufigste Schwachstelle
    if len(password) < MINDESTLAENGE:
        return False
    for zeichen in password:
        if zeichen.isdigit():
            return True
    return False
```

## ANHANG E · WERKZEUG

# Clean-Code-Regeln

Clean Code heißt: Code, den andere ohne Erklärung verstehen und gefahrlos ändern können.

1

## Sprechende Namen

Der Name sagt, was drinsteht oder was passiert:  
`anzahl_fehler` statt `x`, `berechne_summe()`  
 statt `f()`.

2

## Eine Funktion, eine Aufgabe

Wenn du die Aufgabe einer Funktion nur mit „und“ beschreiben kannst, teile sie auf.

3

## Keine magischen Zahlen

Eine nackte `3` im Code erklärt sich nicht.  
`MAX_VERSUCHE = 3` schon, und du änderst sie  
 an einer Stelle.

4

## Nicht wiederholen

Kommt fast derselbe Code mehrmals vor, gehört  
 er in eine Schleife oder eine Funktion.

5

## Kommentare erklären das Warum

Was der Code tut, zeigen gute Namen. Kommen-  
 tare erklären, warum etwas so gelöst ist. Jede  
 Funktion bekommt einen Docstring.

6

## Eingaben prüfen

Rechne damit, dass Nutzende etwas Falsches ein-  
 geben. Frag freundlich erneut, statt abzustürzen.

## Clean-Code-Durchgang am Donnerstag

1. Geh deine eigene Datei mit dem Check unten durch und verbessere sie.
2. Tauscht paarweise: Lies die Datei einer anderen Person. Verstehst du jede Funktion ohne Erklärung? Notiere unklare Stellen und sprech darüber.
3. Startet danach das ganze Programm. Aufräumen darf nichts kaputt machen.

### CLEAN-CODE-CHECK FÜR DEINE DATEI

- ☐ Alle Namen verraten, was drinsteht oder was passiert.
- ☐ Keine Funktion ist länger als etwa 20 Zeilen.
- ☐ Feste Werte stehen als Konstanten oben in der Datei.
- ☐ Kein Codeblock kommt mehrfach fast gleich vor.
- ☐ Jede Funktion hat einen Docstring.
- ☐ Falsche Eingaben führen nicht zum Absturz.
- ☐ Kein auskommentierter alter Code, keine ungenutzten Variablen.
- ☐ „Reformat Code“ in PyCharm ausgeführt, keine gelben Wellenlinien mehr.

## ANHANG E · WERKZEUG

# Clean Code: vorher und nachher

Beide Programme lösen dieselbe Aufgabe. Welches würdest du lieber erweitern?

## VORHER

```
import random
z = random.randint(1, 20)
v = 0
while v < 5:
    e = int(input("Zahl: "))
    v = v + 1
    if e == z:
        print("Richtig!")
        v = 5
    elif e < z:
        print("Zu klein")
        print("Noch", 5 - v, "Versuche")
    else:
        print("Zu groß")
        print("Noch", 5 - v, "Versuche")
```

## NACHHER

```
import random

MAX_VERSUCHE = 5
OBERGRENZE = 20

def gib_hinweis(tipp, geheimzahl):
    """Gibt einen Hinweis-Text zurück."""
    if tipp < geheimzahl:
        return "Zu klein"
    return "Zu groß"

def spiele_runde():
    """Eine Runde Zahlenraten."""
    geheimzahl = random.randint(1, OBERGRENZE)
    for versuch in range(1, MAX_VERSUCHE + 1):
        tipp = int(input("Zahl: "))
        if tipp == geheimzahl:
            print("Richtig!")
            return
        print(gib_hinweis(tipp, geheimzahl))
        verbleibend = MAX_VERSUCHE - versuch
        print("Noch", verbleibend, "Versuche")
    print("Leider verloren. Die Zahl war", geheimzahl)
```

## Was hat sich verbessert?

- **Namen:** `tipp`, `geheimzahl` und `versuch` statt `e`, `z` und `v`.
- **Konstanten** statt magischer Zahlen: Die 5 und die 20 stehen nur noch an einer Stelle.
- **Keine Wiederholung:** Die Zeile „Noch ... Versuche“ steht nur noch einmal.
- **Funktionen** mit Docstring: Jede hat eine klar benannte Aufgabe und lässt sich einzeln testen.
- **Kein Trick mehr:** `v = 5` zum Beenden der Schleife war schwer zu verstehen. `return` sagt klar, dass die Runde vorbei ist.

## ANHANG F · WERKZEUG

# Basics: Arbeiten mit GitHub

Den Code für den Workshop holt ihr euch aus einem GitHub-Repository. Das ist der übliche Weg, wie Teams in der Softwareentwicklung Code teilen.

## Die wichtigsten Begriffe

| BEGRIFF             | BEDEUTUNG                                                                                                               |
|---------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Git</b>          | Ein Programm auf deinem Rechner, das Versionen von Dateien verwaltet. Ist bei euch schon installiert.                   |
| <b>GitHub</b>       | Eine Website, auf der Git-Repositorys gespeichert und geteilt werden.                                                   |
| <b>Repository</b>   | Ein Projektordner mit allen Dateien und ihrer Versionsgeschichte, kurz „Repo“.                                          |
| <b>Clone</b>        | Eine Kopie eines Repositorys von GitHub auf deinen Rechner holen. Das machst du einmal.                                 |
| <b>Pull</b>         | Neue Änderungen von GitHub in deine Kopie holen. Das machst du immer, wenn es etwas Neues gibt.                         |
| <b>Commit, Push</b> | Eigene Änderungen speichern und hochladen. Brauchen wir im Workshop nicht. Im Berufsalltag ist das der nächste Schritt. |

## Das Workshop-Repository holen

1

### Einladung annehmen

Das Repository ist privat. Du bekommst eine Einladung per E-Mail an deine GitHub-Adresse. Klicke dort auf **Accept invitation**. Alternativ findest du sie unter [github.com/notifications](https://github.com/notifications).

2

### Klonen in PyCharm

Startbildschirm: **Clone Repository** (oder Menü **File > New > Project from Version Control**). Links **Git-Hub** wählen und mit deinem Konto anmelden. Dann **Workshop** auswählen, einen Zielordner festlegen und auf **Clone** klicken.

3

### Oder klonen im Terminal

Wechsle in den Ordner, in dem das Repository landen soll, und gib ein:

```
git clone https://github.com/totoongh/Workshop.git
```

Beim ersten Mal öffnet sich ein Anmeldefenster im Browser. Melde dich dort mit deinem GitHub-Konto an.

## ANHANG F · WERKZEUG

# Aktualisieren und Befehle

Wenn der Ausbilder etwas ergänzt, holst du dir die neue Version mit einem Befehl.

## Neue Version holen

### IN PYCHARM

Menü **Git > Update Project** (Windows: Strg + T, macOS: ⌘ + T), dann mit **OK** bestätigen.

### IM TERMINAL

Im Ordner des Repositorys:

```
git pull
```

### WICHTIG: ARBEITE NICHT IM GEKLONTEN ORDNER

Kopiere `projektvorlage` an einen eigenen Ort, bevor du anfängst (Teil 8). Wenn du Dateien im geklonten Ordner änderst, kann `git pull` mit einer Fehlermeldung abbrechen.

## Spickzettel: wichtige Befehle

| BEFEHL                                                   | WAS ER TUT                                                             |
|----------------------------------------------------------|------------------------------------------------------------------------|
| <code>git clone &lt;Adresse&gt;</code>                   | Repository einmalig auf den Rechner holen                              |
| <code>git pull</code>                                    | Neue Änderungen von GitHub holen                                       |
| <code>git status</code>                                  | Zeigen, ob du Dateien im Repository geändert hast                      |
| <code>git log --oneline</code>                           | Die letzten Änderungen kurz auflisten                                  |
| <code>cd &lt;Ordner&gt;</code>                           | In einen Ordner wechseln, <code>cd ..</code> geht eine Ebene nach oben |
| <code>dir</code> (Windows) · <code>ls</code> (macOS)     | Inhalt des aktuellen Ordners anzeigen                                  |
| <code>py datei.py</code> · <code>python3 datei.py</code> | Ein Python-Programm im Terminal starten                                |

Tipp: Im Terminal ergänzt die **Tab-Taste** Ordner- und Dateinamen. Mit **Pfeil nach oben** holst du den letzten Befehl zurück.

## Wenn etwas nicht klappt

| MELDUNG                                     | LÖSUNG                                                                                                                         |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Repository not found                        | Einladung noch nicht angenommen, oder du bist mit einem anderen GitHub-Konto angemeldet.                                       |
| Authentication failed                       | Anmeldung abgelaufen oder abgebrochen. In PyCharm unter <b>Settings &gt; Version Control &gt; GitHub</b> neu anmelden.         |
| Your local changes ... would be overwritten | Du hast im geklonten Ordner etwas geändert. Sichere deine Datei an einem anderen Ort und frag dann im Team oder den Ausbilder. |

## ANHANG G · WERKZEUG

# Bibliotheken: Standardmodule und pip

Niemand programmiert alles selbst. Für viele Aufgaben gibt es fertigen Code, den du in dein Programm einbindest.

## Zwei Arten von Bibliotheken

### STANDARDMODULE

**Sind in Python schon enthalten.** Du bindest sie mit `import` ein und kannst sie sofort nutzen.

Beispiele: `random`, `math`, `time`, `datetime`, `string`

Für alle im Workshop. Mindestens ein Modul gehört in euer Programm.

### EXTERNE PAKETE (PIP)

**Musst du vorher installieren.** Sie kommen aus dem Python Package Index (PyPI), einer Sammlung von über 500.000 Paketen.

Beispiele: `rich`, `pyfiglet`, `pygame`

Exkurs für Fortgeschrittene, siehe nächste Seite.

## So bindest du ein Modul ein

```
import random                # ganzes Modul: random.randint(1, 6)
from random import randint    # nur eine Funktion: randint(1, 6)
import datetime as dt         # mit Kurznamen: dt.date.today()
```

Am verständlichsten ist meist die erste Form. Man sieht beim Lesen sofort, woher eine Funktion kommt.

## Nützliche Standardmodule

| MODUL                 | BEISPIEL                                                                                                     | WOFÜR                                            |
|-----------------------|--------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| <code>random</code>   | <code>random.randint(1, 6)</code><br><code>random.choice(liste)</code><br><code>random.shuffle(liste)</code> | Zufallszahl, zufälliges Element, Liste mischen   |
| <code>math</code>     | <code>math.sqrt(16)</code> · <code>math.ceil(4.2)</code>                                                     | Wurzel, Aufrunden, <code>math.pi</code>          |
| <code>time</code>     | <code>time.sleep(1)</code>                                                                                   | Eine Sekunde warten, z. B. für Spannung im Spiel |
| <code>datetime</code> | <code>date.today()</code> nach<br><code>from datetime import date</code>                                     | Heutiges Datum, mit Daten rechnen                |
| <code>string</code>   | <code>string.ascii_letters</code><br><code>string.digits</code>                                              | Fertige Zeichenvorräte, z. B. für Passwörter     |

### SELBST NACHSCHLAGEN

`help(random.randint)` in der Python-Konsole zeigt die Beschreibung einer Funktion. Alle Standardmodule sind unter [docs.python.org/3/library](https://docs.python.org/3/library) beschrieben. In PyCharm zeigt `Strg + Q` (macOS: `F1`) die Doku zu der Funktion unter dem Cursor.

Beispielprogramm im Repository: `beispiele/01_standardmodule.py`

## ANHANG G · EXKURS FÜR FORTGESCHRITTENE

# Externe Pakete mit pip

pip ist das Installationsprogramm für Python-Pakete. Es lädt ein Paket aus dem Python Package Index und installiert es für deinen Python-Interpreter.

## Ein Paket installieren

### IN PYCHARM

Unten links das Fenster **Python Packages** öffnen, den Paketnamen suchen, z. B. `rich`, und auf **Install** klicken.

Alternativ: **Settings > Project > Python Interpreter**, dann auf **+** klicken.

### IM TERMINAL (AUCH IN PYCHARM UNTEN)

```
py -m pip install rich
```

macOS: `python3 -m pip install rich`

Prüfen, was installiert ist: `py -m pip list`

## Ein Paket nutzen

```
from rich.console import Console
```

```
konsole = Console()
```

```
konsole.print("Hallo [bold green]Workshop[/bold green]!")
```

Beispielprogramm mit Tabelle im Repository: `beispiele/02_pip_rich.py`

## Pakete für eure Projekte

| PAKET                 | WOFÜR                                                                   |
|-----------------------|-------------------------------------------------------------------------|
| <code>rich</code>     | Farbige Ausgaben, Tabellen und Fortschrittsbalken in der Konsole        |
| <code>pyfiglet</code> | Große Überschriften aus Textzeichen, z. B. für einen Spieletitel        |
| <code>colorama</code> | Einfache Textfarben in der Konsole, auch unter Windows                  |
| <code>pygame</code>   | Spiele mit Grafik und Tastatursteuerung. Nur für sehr Fortgeschrittene. |

### DARAUF MUSST DU ACHTEN

- **Alle im Team brauchen das Paket.** Notiert es in einer Datei `requirements.txt` im Projektordner, ein Paket pro Zeile. Dann installiert jede Person alles mit `py -m pip install -r requirements.txt`.
- **Nur bekannte Pakete installieren.** Achte auf die genaue Schreibweise. Es gibt gefälschte Pakete mit ähnlichen Namen.
- **Klappt die Installation nicht**, zum Beispiel wegen des Firmennetzes, frag den Ausbilder. Bastle nicht an den Netzwerkeinstellungen.
- PyCharm legt für jedes Projekt eine eigene Umgebung an ( `.venv` ). Ein Paket, das du in einem Projekt installierst, fehlt deshalb im nächsten Projekt.

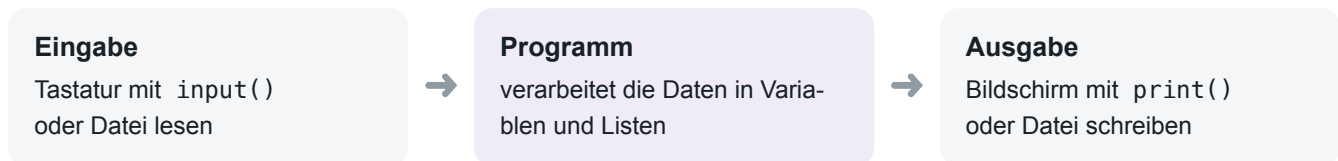
## ANHANG H · EXKURS FÜR FORTGESCHRITTENE

# Dateien und Datenströme

Bisher vergisst euer Programm alles, sobald es endet. Mit Dateien bleiben Daten erhalten: Highscores, Inventar, Tickets.

## Die Idee: Daten fließen

Ein Programm bekommt Daten von irgendwoher und gibt Daten irgendwohin ab. Man spricht von **Datenströmen**, weil die Daten Zeichen für Zeichen hinein- und hinausfließen. Bisher kennst du zwei Ströme. Dateien sind zwei weitere.



Der Unterschied: Was auf dem Bildschirm steht, ist nach dem Programmende weg. Was in einer Datei auf der Festplatte steht, bleibt.

## Eine Datei öffnen

```
with open("notizen.txt", "r", encoding="utf-8") as datei:
    inhalt = datei.read()
    print(inhalt)
```

- `open()` öffnet die Datei und gibt ein Dateiojekt zurück. Hier heißt es `datei`.
- `with ... as ...:` sorgt dafür, dass die Datei am Ende des eingerückten Blocks automatisch geschlossen wird, auch wenn ein Fehler passiert.
- `encoding="utf-8"` sorgt dafür, dass Umlaute auf jedem Rechner richtig ankommen.

| MODUS | BEDEUTUNG         | ACHTUNG                                                  |
|-------|-------------------|----------------------------------------------------------|
| "r"   | lesen (read)      | Fehler, wenn die Datei nicht existiert                   |
| "w"   | schreiben (write) | Legt die Datei neu an. Vorhandener Inhalt wird gelöscht! |
| "a"   | anhängen (append) | Schreibt ans Ende, legt die Datei bei Bedarf an          |

## ANHANG H · BASICS

# Dateien: die Grundlagen zum Nachschlagen

Die wichtigsten Bausteine. Alle Beispiele findest du lauffähig im Ordner `beispiele`.

## Schreiben, anhängen und lesen

```
import os

DATEI = "liste.txt"

with open(DATEI, "w", encoding="utf-8") as datei:    # neu anlegen
    datei.write("Laptop\n")
    datei.write("Headset\n")

with open(DATEI, "a", encoding="utf-8") as datei:    # ans Ende anhängen
    datei.write("Maus\n")

geraete = []
if os.path.exists(DATEI):                            # gibt es die Datei?
    with open(DATEI, "r", encoding="utf-8") as datei:
        for zeile in datei:
            geraete.append(zeile.strip())
print(geraete)                                       # ['Laptop', 'Headset', 'Maus']
```

- `write()` macht keinen Zeilenumbruch. Den schreibst du selbst mit `\n`. Zahlen wandelst du vorher mit `str()` um.
- `strip()` entfernt beim Lesen den Zeilenumbruch am Ende jeder Zeile.
- `os.path.exists()` prüft vorher, ob es die Datei gibt. Sonst bricht `"r"` mit einem Fehler ab.

## Mehrere Werte pro Zeile: Trennzeichen

Speichere zusammengehörige Werte in einer Zeile, getrennt durch ein Semikolon. Beim Lesen zerlegt `split(";")` die Zeile wieder in eine Liste.

```
def speichere_highscore(name, punkte):
    with open("highscores.txt", "a", encoding="utf-8") as datei:
        datei.write(name + ";" + str(punkte) + "\n")

# Zeile in der Datei: Ada;42
teile = "Ada;42".split(";")    # ['Ada', '42']
punkte = int(teile[1])         # 42
```

Ganzes Beispiel mit Laden und Anzeigen: `beispiele/04_highscore.py`. Für Listen mit Dictionaries gibt es das Modul `json`: `beispiele/05_json_speichern.py`.

## Typische Stolpersteine

| PROBLEM                        | URSACHE UND LÖSUNG                                                                                                       |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <code>FileNotFoundError</code> | Die Datei gibt es (noch) nicht, oder sie liegt in einem anderen Ordner. Vorher mit <code>os.path.exists()</code> prüfen. |
| Datei ist plötzlich leer       | Mit <code>"w"</code> statt <code>"a"</code> geöffnet. <code>"w"</code> löscht den alten Inhalt.                          |
| Alles steht in einer Zeile     | <code>\n</code> am Ende von <code>write()</code> vergessen.                                                              |
| Umlaute sehen kaputt aus       | <code>encoding="utf-8"</code> fehlt beim Schreiben oder Lesen.                                                           |

## ANHANG I · EXKURS FÜR FORTGESCHRITTENE

# Grafische Oberflächen mit tkinter

Statt Text im Terminal: ein Fenster mit Eingabefeldern und Knöpfen. tkinter ist in Python enthalten und reicht für kleine Werkzeuge und Spiele.

## Die Idee: Das Programm wartet auf Ereignisse

### KONSOLEPROGRAMM

Das Programm bestimmt den Ablauf. Es läuft von oben nach unten und wartet bei `input()`, bis du etwas eingibst.

### PROGRAMM MIT OBERFLÄCHE

Die Nutzerin oder der Nutzer bestimmt den Ablauf. Das Programm baut das Fenster auf und wartet dann in `mainloop()` auf **Ereignisse**, zum Beispiel einen Klick. Bei einem Klick ruft es die Funktion auf, die zu diesem Knopf gehört.

## Ein erstes Fenster

```
import tkinter as tk

def begruessen():
    name = eingabe.get()
    ausgabe.config(text="Hallo " + name + "!")

fenster = tk.Tk()
fenster.title("Mein erstes Fenster")

tk.Label(fenster, text="Wie heißt du?").pack(padx=20, pady=(20, 5))
eingabe = tk.Entry(fenster)
eingabe.pack(padx=20)
tk.Button(fenster, text="Begrüßen", command=begruessen).pack(pady=10)
ausgabe = tk.Label(fenster, text="")
ausgabe.pack(padx=20, pady=(0, 20))

fenster.mainloop()
```

1. `tk.Tk()` erzeugt das Fenster.
2. Jedes Element (Widget) bekommt das Fenster als ersten Parameter. `pack()` setzt es untereinander ins Fenster.
3. `command=begruessen` verbindet den Knopf mit der Funktion. **Ohne Klammern!** Mit Klammern würde die Funktion sofort einmal aufgerufen.
4. `fenster.mainloop()` zeigt das Fenster und wartet auf Ereignisse. Code danach läuft erst, wenn das Fenster geschlossen ist.

Lauffähig im Repository: `beispiele/06_gui_minimal.py`

## ANHANG I · BASICS

# tkinter: die Grundlagen zum Nachschlagen

Die wichtigsten Widgets und ein Tipp, der dir viel Arbeit spart.

## Wichtige Widgets

| WIDGET     | ERZEUGEN                                              | WOFÜR                                                                                         |
|------------|-------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Label      | <code>tk.Label(fenster, text="...")</code>            | Text anzeigen. Ändern mit <code>.config(text="...")</code>                                    |
| Entry      | <code>tk.Entry(fenster, width=10)</code>              | Einzeiliges Eingabefeld. Lesen mit <code>.get()</code> , liefert immer einen <code>str</code> |
| Button     | <code>tk.Button(fenster, text="OK", command=f)</code> | Knopf, ruft beim Klick die Funktion <code>f</code> auf                                        |
| Listbox    | <code>tk.Listbox(fenster)</code>                      | Liste anzeigen. Einfügen mit <code>.insert(tk.END, "...")</code>                              |
| messagebox | <code>messagebox.showinfo("Titel", "Text")</code>     | Hinweisfenster. Vorher: <code>from tkinter import messagebox</code>                           |

## Anordnen mit grid()

Statt `pack()` kannst du Widgets mit `grid(row=0, column=1)` in Zeilen und Spalten anordnen, zum Beispiel Beschriftung links und Eingabefeld rechts. Verwende in einem Fenster entweder `pack()` oder `grid()`, nicht beides. Ein vollständiges Beispiel mit `grid()` steht in `beispiele/07_gui_passwort.py`.

### DER WICHTIGSTE TIPP: LOGIK UND OBERFLÄCHE TRENNEN

Programmiere dein Feature zuerst als Konsolenversion mit einer Funktion, die etwas berechnet und mit `return` zurückgibt. Die Oberfläche ruft dann nur noch diese Funktion auf. So kannst du die Logik ohne Fenster testen und deine bisherige Arbeit weiterverwenden.

```
def erzeuge_passwort(laenge):           # Logik: kennt kein Fenster
    ...
    return password

def knopf_geklickt():                   # Oberfläche: liest, ruft auf, zeigt an
    laenge = int(eingabe_laenge.get())
    ausgabe.config(text=erzeuge_passwort(laenge))
```

Vollständiger Passwort-Generator mit Prüfung der Eingabe: `beispiele/07_gui_passwort.py`

## Typische Stolpersteine

| PROBLEM                                            | URSACHE UND LÖSUNG                                                 |
|----------------------------------------------------|--------------------------------------------------------------------|
| Funktion läuft sofort beim Start, Klick tut nichts | <code>command=f()</code> statt <code>command=f</code>              |
| Fenster erscheint nicht                            | <code>fenster.mainloop()</code> fehlt am Ende                      |
| Rechnen mit der Eingabe klappt nicht               | <code>.get()</code> liefert Text. Mit <code>int()</code> umwandeln |